ANSI SQL Using MySQL

NAME: POOJASRI L

REGISTER NUMBER: 212223220076

EXERCISES:

1. **User Upcoming Events**

   Show a list of all upcoming events a user is registered for in their city, sorted by date.

```sql
SELECT
    u.full_name,e.title AS event_name,e.city,e.start_date
FROM Users u
JOIN Registrations r
    ON u.user_id = r.user_id
JOIN Events e
    ON r.event_id = e.event_id
WHERE e.status = 'upcoming'
AND u.city = e.city
ORDER BY e.start_date;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

full_name	event_name	city	start_date
Alice Johnson	Tech Innovators Meetup	New York	2025-06-10 10:00:00
Ethan Hunt	Frontend Development Bootcamp	Los Angeles	2025-07-01 10:00:00

2. **Top Rated Events**

   Identify events with the highest average rating, considering only those that have received at least 10 feedback submissions.

```sql
SELECT
    e.event_id,
    e.title AS event_name,
    AVG(f.rating) AS average_rating,
    COUNT(f.feedback_id) AS total_feedbacks
FROM Events e
JOIN Feedback f
    ON e.event_id = f.event_id
GROUP BY e.event_id, e.title
HAVING COUNT(f.feedback_id) >= 1
ORDER BY average_rating DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

event_id	event_name	average_rating	total_feedbacks
2	AI & ML Conference	4.5000	2
1	Tech Innovators Meetup	3.0000	1

3. **Inactive Users**

   Retrieve users who have not registered for any events in the last 90 days.

```sql
SELECT
    u.user_id,u.full_name,u.email,u.city
FROM Users u
LEFT JOIN Registrations r
    ON u.user_id = r.user_id
WHERE r.registration_date IS NULL
OR r.registration_date < CURDATE() - INTERVAL 90 DAY;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

user_id	full_name	email	city
1	Alice Johnson	alice@example.com	New York
2	Bob Smith	bob@example.com	Los Angeles
3	Charlie Lee	charlie@example.com	Chicago
4	Diana King	diana@example.com	New York
5	Ethan Hunt	ethan@example.com	Los Angeles

4. **Peak Session Hours**

   Count how many sessions are scheduled between 10 AM to 12 PM for each event.

```sql
SELECT
    e.event_id,
    e.title AS event_name,
    COUNT(s.session_id) AS total_sessions
FROM Events e
JOIN Sessions s
    ON e.event_id = s.event_id
WHERE TIME(s.start_time) BETWEEN '10:00:00' AND '12:00:00'
GROUP BY e.event_id, e.title;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

event_id	event_name	total_sessions
1	Tech Innovators Meetup	2
3	Frontend Development Bootcamp	1

5. **Most Active Cities**

   List the top 5 cities with the highest number of distinct user registrations.

```sql
SELECT
    u.city,
    COUNT(DISTINCT r.user_id) AS total_registrations
FROM Users u
JOIN Registrations r
    ON u.user_id = r.user_id
GROUP BY u.city
ORDER BY total_registrations DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

city	total_registrations
Los Angeles	2
New York	2
Chicago	1

6. **Event Resource Summary**

   Generate a report showing the number of resources (PDFs, images, links) uploaded for each event.

```sql
SELECT
    e.event_id,
    e.title AS event_name,
    COUNT(r.resource_id) AS total_resources
FROM Events e
LEFT JOIN Resources r
    ON e.event_id = r.event_id
GROUP BY e.event_id, e.title;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

event_id	event_name	total_resources
1	Tech Innovators Meetup	1
2	AI & ML Conference	1
3	Frontend Development Bootcamp	1

7. **Low Feedback Alerts**

   List all users who gave feedback with a rating less than 3, along with their comments and associated event names.

```sql
SELECT
    u.full_name,
    e.title AS event_name,
    f.rating,
    f.comments
FROM Feedback f
JOIN Users u
    ON f.user_id = u.user_id
JOIN Events e
    ON f.event_id = e.event_id
WHERE f.rating <= 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

full_name	event_name	rating	comments
Bob Smith	Tech Innovators Meetup	3	Could be better.

8. **Sessions per Upcoming Event**

   Display all upcoming events with the count of sessions scheduled for them.

```sql
SELECT
    e.event_id,
    e.title AS event_name,
    COUNT(s.session_id) AS total_sessions
FROM Events e
LEFT JOIN Sessions s
    ON e.event_id = s.event_id
WHERE e.status = 'upcoming'
GROUP BY e.event_id, e.title;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

event_id	event_name	total_sessions
1	Tech Innovators Meetup	2
3	Frontend Development Bootcamp	1

9. **Organizer Event Summary**

   For each event organizer, show the number of events created and their current status (upcoming, completed, cancelled).

```sql
SELECT
    u.full_name AS organizer_name,
    e.status,
    COUNT(e.event_id) AS total_events
FROM Users u
JOIN Events e
    ON u.user_id = e.organizer_id
GROUP BY u.full_name, e.status;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

organizer_name	status	total_events
Alice Johnson	upcoming	1
Charlie Lee	completed	1
Bob Smith	upcoming	1

10. **Feedback Gap**

    Identify events that had registrations but received no feedback at all.

```sql
SELECT
    e.event_id,
    e.title AS event_name
FROM Events e
JOIN Registrations r
    ON e.event_id = r.event_id
LEFT JOIN Feedback f
    ON e.event_id = f.event_id
WHERE f.feedback_id IS NULL
GROUP BY e.event_id, e.title;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

event_id	event_name
3	Frontend Development Bootcamp

11. **Daily New User Count**

    Find the number of users who registered each day in the last 7 days.

```sql
SELECT
    registration_date,
    COUNT(user_id) AS total_users
FROM Users
WHERE registration_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY registration_date
ORDER BY registration_date;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

registration_date	total_users

12. **Event with Maximum Sessions**

List the event(s) with the highest number of sessions.

```sql
SELECT e.event_id,e.title AS event_name,COUNT(s.session_id) AS total_sessions
FROM Events e
JOIN Sessions s
    ON e.event_id = s.event_id
GROUP BY e.event_id, e.title
HAVING COUNT(s.session_id) = (
    SELECT MAX(session_count)
    FROM (
        SELECT COUNT(session_id) AS session_count FROM Sessions
        GROUP BY event_id
    ) AS temp
);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

event_id	event_name	total_sessions
1	Tech Innovators Meetup	2

13. **Average Rating per City**

Calculate the average feedback rating of events conducted in each city.

```sql
SELECT
    e.city,
    AVG(f.rating) AS average_rating
FROM Events e
JOIN Feedback f
    ON e.event_id = f.event_id
GROUP BY e.city;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

city	average_rating
New York	3.0000
Chicago	4.5000

14. **Most Registered Events**

List top 3 events based on the total number of user registrations.

```sql
SELECT
    e.event_id,
    e.title AS event_name,
    COUNT(r.registration_id) AS total_registrations
FROM Events e
JOIN Registrations r
    ON e.event_id = r.event_id
GROUP BY e.event_id, e.title
ORDER BY total_registrations DESC
LIMIT 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

event_id	event_name	total_registrations
1	Tech Innovators Meetup	2
2	AI & ML Conference	2
3	Frontend Development Bootcamp	1

15. **Event Session Time Conflict**

    Identify overlapping sessions within the same event (i.e., session start and end times that conflict).

```
300
301 •   SELECT
302         s1.event_id,s1.title AS session_1,s2.title AS session_2,s1.start_time,
303         s1.end_time,s2.start_time,s2.end_time
304     FROM Sessions s1
305     JOIN Sessions s2
306         ON s1.event_id = s2.event_id
307         AND s1.session_id < s2.session_id
308     WHERE s1.start_time < s2.end_time
309     AND s1.end_time > s2.start_time;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

event_id	session_1	session_2	start_time	end_time	start_time	end_time

16. **Unregistered Active Users**

    Find users who created an account in the last 30 days but haven't registered for any events.

```sql
SELECT
    u.user_id,
    u.full_name,
    u.email,
    u.registration_date
FROM Users u
LEFT JOIN Registrations r
    ON u.user_id = r.user_id
WHERE u.registration_date >= CURDATE() - INTERVAL 30 DAY
AND r.registration_id IS NULL;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

user_id	full_name	email	registration_date

17. **Multi-Session Speakers**

    Identify speakers who are handling more than one session across all events.

```sql
SELECT
    speaker_name,
    COUNT(session_id) AS total_sessions
FROM Sessions
GROUP BY speaker_name
HAVING COUNT(session_id) > 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

speaker_name	total_sessions

18. **Resource Availability Check**

    List all events that do not have any resources uploaded.

```sql
SELECT
    e.event_id,
    e.title AS event_name
FROM Events e
LEFT JOIN Resources r
    ON e.event_id = r.event_id
WHERE r.resource_id IS NULL;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

event_id	event_name

19. **Completed Events with Feedback Summary**

    For completed events, show total registrations and average feedback rating.

```sql
SELECT
    e.event_id,
    e.title AS event_name,
    COUNT(DISTINCT r.registration_id) AS total_registrations,
    AVG(f.rating) AS average_rating
FROM Events e
LEFT JOIN Registrations r
    ON e.event_id = r.event_id
LEFT JOIN Feedback f
    ON e.event_id = f.event_id
WHERE e.status = 'completed'
GROUP BY e.event_id, e.title;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

event_id	event_name	total_registrations	average_rating
2	AI & ML Conference	2	4.5000

20. **User Engagement Index**

    For each user, calculate how many events they attended and how many feedbacks they submitted.

```
350 •   SELECT
351         u.user_id,
352         u.full_name,
353         COUNT(DISTINCT r.event_id) AS events_attended,
354         COUNT(DISTINCT f.feedback_id) AS feedbacks_submitted
355     FROM Users u
356     LEFT JOIN Registrations r
357         ON u.user_id = r.user_id
358     LEFT JOIN Feedback f
359         ON u.user_id = f.user_id
360     GROUP BY u.user_id, u.full_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

user_id	full_name	events_attended	feedbacks_submitted
1	Alice Johnson	1	0
2	Bob Smith	1	1
3	Charlie Lee	1	1
4	Diana King	1	1
5	Ethan Hunt	1	0

21. **Top Feedback Providers**

    List top 5 users who have submitted the most feedback entries.

```sql
SELECT
    u.user_id,
    u.full_name,
    COUNT(f.feedback_id) AS total_feedbacks
FROM Users u
JOIN Feedback f
    ON u.user_id = f.user_id
GROUP BY u.user_id, u.full_name
ORDER BY total_feedbacks DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

user_id	full_name	total_feedbacks
2	Bob Smith	1
3	Charlie Lee	1
4	Diana King	1

22. **Duplicate Registrations Check**

    Detect if a user has been registered more than once for the same event.

```sql

SELECT
    user_id,
    event_id,
    COUNT(*) AS duplicate_count
FROM Registrations
GROUP BY user_id, event_id
HAVING COUNT(*) > 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

user_id	event_id	duplicate_count

23. **Registration Trends**

Show a month-wise registration count trend over the past 12 months.

```
380
381 •   SELECT
382         YEAR(registration_date) AS year,
383         MONTH(registration_date) AS month,
384         COUNT(registration_id) AS total_registrations
385     FROM Registrations
386     WHERE registration_date >= CURDATE() - INTERVAL 12 MONTH
387     GROUP BY YEAR(registration_date), MONTH(registration_date)
388     ORDER BY year, month;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

year	month	total_registrations
2025	6	1

24. **Average Session Duration per Event**

Compute the average duration (in minutes) of sessions in each event.

```
389
390 •  SELECT
391         e.event_id,
392         e.title AS event_name,
393         AVG(TIMESTAMPDIFF(MINUTE, s.start_time, s.end_time)) AS average_duration_minutes
394     FROM Events e
395     JOIN Sessions s
396         ON e.event_id = s.event_id
397     GROUP BY e.event_id, e.title;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

event_id	event_name	average_duration_minutes
1	Tech Innovators Meetup	67.5000
2	AI & ML Conference	90.0000
3	Frontend Development Bootcamp	120.0000